极其庞大的神经网络：稀疏门控的专家混合层

诺姆·沙齐尔[1], Azalia Mirhoseini*†[1],Krzysztof Maziarz*[2],Andy Davis[1],QuocLe[1],Geoffrey Hinton[1] 和 J eff Dean[1]

[1]Google Brain，{noam,azalia,andydavis,qvl,geoffhinton,jeff}@google.com [2]雅盖隆大学，克拉科夫，krzysztof.maziarz@student.uj.edu.pl

摘要

神经网络吸收信息的容量受其参数数量的限制。条件计算指的是网络的部分子模块按每个样本激活；从理论上看，它被提出为一种在不按比例增加计算的情况下显著提升模型容量的方法。然而在实践中，仍存在重大的算法与性能挑战。在本论文中，我们解决了这些挑战，最终兑现了条件计算的承诺：在现代 GPU 集群上，仅以极小的计算效率损失，便将模型容量提升了超过1000倍。我们提出了一种稀疏门控的专家混合层（专家混合），由多达数千个前馈子网络组成。一个可训练的门控网络为每个样本选择这些专家的一个稀疏组合。我们将该专家混合应用于语言建模与机器翻译任务，在这些任务中，模型容量对于吸收训练语料库中海量知识至关重要。我们提出了模型架构，其中在堆叠的 LSTM 层之间以卷积方式应用一个参数量高达 1370 亿的专家混合。在大规模语言建模与机器翻译基准上，这些模型以更低的计算成本取得了显著优于当前最先进水平的结果。

1 引言与相关工作

1.1 条件计算

在训练数据和模型规模两个方面利用缩放，一直是深度学习成功的核心。当数据集足够大时，增加神经网络的容量（参数数量）可以带来更高的预测准确率。这已经在文本（Sutskever et al., 2014; Bahdanau et al., 2014; Jozefowicz et al., 2016年; Wu et al., 2016年）、图像（Krizhevsky et al., 2012; Le et al., 2012）以及音频（Hinton et al., 2012; Amodei et al., 2015年）等领域得到证明。对于典型的深度学习模型，每个样本都会激活整个模型；随着模型规模和训练样本数量同时增加，训练成本会大致呈二次增长。不幸的是，计算能力和分布式计算的进步仍难以满足这种需求。

已经有多种形式的条件计算被提出，作为在不按比例增加计算成本的情况下提升模型容量的方法（Davis & Arel，2013年；Bengio et al.，2013年；Eigen et al.，2013年；Ludovic Denoyer，2014年；Cho & Bengio，2014年；Bengio et al.，2015年；Almahairi et al.，2015年）。在这些方案中，网络的大部分组件会按每个样例分别决定激活或不激活。门控决策可以是二元的，或稀疏且连续的，亦可为随机或确定性的。已经提出使用各种形式的强化学习和反向传播来训练这些门控决策。

*共同主要贡献者

†作为 Google Brain Residency 项目成员完成的工作（g.co/brainresidency）

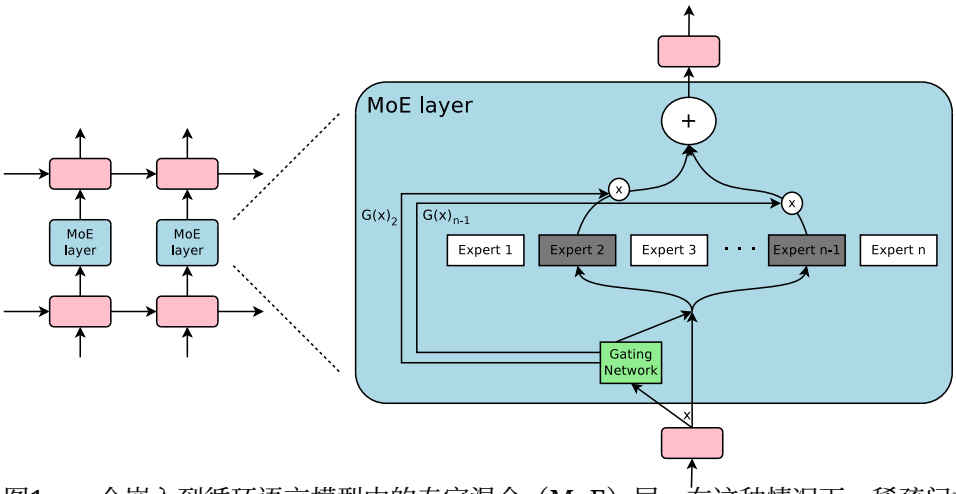

图1：一个嵌入到循环语言模型中的专家混合（MoE）层。在这种情况下，稀疏门控函数选择两个专家来执行计算。它们的输出由门控网络的输出调制。

尽管理论上这些想法很有前景，但迄今为止还没有工作能够展示在模型容量、训练时间或模型质量方面的巨大改进。我们将其归因于以下挑战的综合作用：

- 现代计算设备，尤其是 GPU，在算术运算方面远快于分支跳转。上述多数工作认识到这一点，并提出在每次门控决策时打开/关闭网络的大块部分。
- 较大的批大小对性能至关重要，因为它们可以摊薄参数传输与更新的成本。条件计算会降低网络中按条件激活的块的批大小。
- 网络带宽可能成为瓶颈。一个 GPU 集群的计算能力可能比设备间总网络带宽高出数千倍。要实现较高的计算效率，算法中计算需求相对于网络需求的比例必须超过这一比值。嵌入层可被视为一种条件计算，正是因此而受到这一问题的掣肘。由于嵌入通常需要通过网络发送，（样本、参数）交互的数量受限于网络带宽，而不是计算能力。
- 根据具体方案，可能需要引入损失项，以在每个块和/或每个样本上达到期望的稀疏性水平。Bengio 等人（2015年）使用了三个此类项。这些问题会同时影响模型质量和负载均衡。
- 对于非常大的数据集，模型容量至关重要。关于条件计算的现有文献处理的都是相对较小的图像识别数据集，规模最多为 600,000 张图像。很难想象，这些图像的标签能提供足够的信号来充分训练一个拥有数百万——更不用说数十亿——参数的模型。

在这项工作中，我们首次解决了上述所有挑战，最终兑现了条件计算的承诺。我们在模型容量上获得了超过 1000 倍的提升，而计算效率仅有轻微损失，并在公开的语言建模和翻译数据集上显著推进了最先进结果。

1.2 我们的方法：稀疏门控专家混合层

我们对条件计算的处理方法是引入一种新的通用神经网络组件：稀疏门控的专家混合层（MoE）。该 专家混合 由若干 专家 组成，每个 专家 都是一个简单的前馈神经网络，以及一个可训练的门控网络，用于为每个 输入 选择稀疏的 专家 组合（见图1）。网络的所有部分通过反向传播联合训练。

尽管所引入的 技术 是通用的，在本文中我们聚焦于 语言建模 和 机器翻译 任务，这些任务已知能从非常 大规模模型 中受益。具体而言，我们将 专家混合 以卷积方式应用在堆叠的 LSTM 层之间（Hochreiter & Schmidhuber, 1997），如图1所示。对于文本中的每个位置调用一次该 专家混合，在每个位置选择可能不同的 专家 组合。不同的 专家 往往会基于句法和语义变得高度专门化（见附录 E 表9）。在 语言建模 和 机器翻译 基准 上，我们以其一小部分的 计算成本 改进了已发表的最佳结果。

1.3 关于专家混合的相关工作

自二十多年前提出以来（Jacobs 等，1991；Jordan & Jacobs，1994），专家混合这一方法一直是大量研究的对象。已经提出了不同类型的专家架构，例如 SVMs（Collobert 等，2002）、高斯过程（Tresp，2001；Theis & Bethge，2015年；Deisenroth & Ng，2015年）、狄利克雷过程（Shahbaba & Neal，2009）以及深度网络。其他工作则聚焦于不同的专家配置，例如层次结构（Yao 等，2009）、无限数量的专家（Rasmussen & Ghahramani，2002），以及按顺序添加专家（Aljundi 等，2016年）。Garmash & Monz（2016年）提出了一种用于机器翻译、以专家混合为形式的集成模型。门控网络在一个预训练的集成 NMT 模型上进行训练。

上述工作关注的是顶层的专家混合。也就是说，专家混合即整个模型。Eigen 等人（2013）提出了将多个拥有各自门控网络的 MoE 作为深度模型组成部分的想法。直观上，后一种方法更强大，因为复杂问题可能包含许多子问题，而每个子问题都需要不同的专家。他们还在结论中提到引入稀疏性的潜力，从而将 MoE 变成一种用于计算的载体。

我们的工作基于这种将 MoE 作为通用神经网络组件的用法。虽然 Eigen 等人（2013）使用两个堆叠的 MoE，从而进行两次门控决策，但我们将 MoE 应用于卷积，使得文本中的每个位置都可以做出不同的门控决策。我们还实现了稀疏门控，并证明其是大幅提升模型容量的一种切实可行的方法。

2 专家混合（MoE）层的结构

专家混合（MoE）层由一组 n “专家网络” $E_1, \cdots, E_n$，以及一个“门控网络” G 组成，其输出是一个稀疏的 n维向量。图1展示了该专家混合（MoE）模块的概览。这些专家本身就是神经网络，每个都有各自的参数。尽管原则上我们只要求各专家接受相同尺寸的输入并产生相同尺寸的输出，但在本文的初步研究中，我们将范围限制在这样一种情况：这些模型都是具有相同架构但参数独立的前馈网络。

设对于给定的输入 x，用 $G(x)$ 和 $E_i(x)$ 分别表示门控网络的输出与第 i 个专家网络的输出。专家混合模块的输出 y 可写为：

$$y = \sum_{i=1}^{n} G(x)_i E_i(x) \tag{1}$$

我们基于 $G(x)$ 的输出稀疏性来节省计算。凡是 $G(x)_i = 0$ 的地方，我们无需计算 $E_i(x)$. 在我们的实验中，我们最多有数千个专家，但对每个样本只需评估少数几个。如果专家数量非常大，我们可以通过使用两级分层专家混合来降低分支因子。在分层专家混合中，主门控网络选择“专家”的稀疏加权组合，其中每个“专家”本身都是带有自身门控网络的次级专家混合。下文我们聚焦于普通的专家混合。关于分层专家混合的更多细节见附录 B。

我们的实现与其他条件计算的模型相关。其专家为简单权重矩阵的专家混合与（Cho & Bengio，2014年）提出的参数化权重矩阵相似。其专家具有一个隐藏层的专家混合与（Bengio 等，2015年）中描述的分块式 Dropout（随机失活）相似，其中被 Dropout（随机失活）的层夹在完全激活的层之间。

2.1 门控网络

Softmax 门控：一种非稀疏门控函数的简单选择（Jordan & Jacobs，1994）是将输入乘以一个可训练的权重矩阵 W_g 然后再应用 $Softmax$ 函数。

$$G_\sigma(x) = Softmax(x \cdot W_g) \tag{2}$$

带噪声的 Top-k 门控： 我们在 Softmax 门控网络中加入两个组件：稀疏性和噪声。在取 Softmax 之前，我们加入可调的高斯噪声，然后只保留 Top-k 值，将其余值设为 $-\infty$（这会使相应的门控值为 0）。如上所述，稀疏性用于节省计算。尽管理论上这种稀疏形式会在门控函数的输出中产生令人担忧的不连续性，但在实践中我们尚未观察到这是个问题。噪声项有助于负载均衡，这将在附录A中讨论。每个组件的噪声量由第二个可训练的权重矩阵 W_{noise}控制。

$$G(x) = Softmax(KeepTopK(H(x), k)) \tag{3}$$

$$H(x)_i = (x \cdot W_g)_i + StandardNormal() \cdot Softplus((x \cdot W_{noise})_i) \tag{4}$$

$$KeepTopK(v, k)_i = \begin{cases} v_i & \text{if } v_i \text{ is in the top } k \text{ elements of } v. \\ -\infty & \text{otherwise.} \end{cases} \tag{5}$$

训练门控网络 我们通过简单的反向传播来训练门控网络，与模型的其余部分一同训练。如果我们选择 $k > 1$，则 Top-k 专家的门控值相对于门控网络的权重具有非零导数。这种偶尔敏感的行为在（Bengio 等，2013年）中已就带噪声的整流器进行描述。梯度也通过门控网络反向传播到其输入。我们的方法在此不同于（Bengio 等，2015年），他们使用布尔门控和 REINFORCE 风格的方法来训练门控网络。

3 A应对性能挑战 NGES

3.1 批次缩小问题

在现代 CPU 和 GPU 上，为了计算效率，需要较大的批大小，以摊销参数加载和更新的开销。如果门控网络为每个样本从n 个专家中选择 k 个，那么对于包含 b 个样本的批次，每个专家仅接收一个更小的、约$\frac{kb}{n} \ll b$ 个样本的批次。这会导致随着专家数量的增加，朴素的专家混合实现变得非常低效。解决这一批次缩小问题的方法是尽可能增大原始批大小。然而，批大小往往受限于在前向传播与反向传播之间存储激活所需的内存。我们提出以下技术来增大批大小：

混合数据并行与模型并行： 在常规的分布式训练设置中，不同设备上的多个模型副本以异步方式处理不同的数据批次，并通过一组参数服务器来同步参数。在我们的技术中，这些不同的批次以同步方式运行，以便为MoE层进行合并。我们按照常规的数据并行方案分布模型的标准层和门控网络，但每个专家只保留一个共享副本。MoE层中的每个专家接收一个合并后的批次，该批次由所有数据并行输入批次中的相关样本组成。同一组设备既充当数据并行副本（用于标准层和门控网络），又充当模型并行分片（每个分片托管一部分专家）。如果模型分布在 d台设备上，并且每台设备处理一个批大小为 b的批次，则每个专家接收的批次大约包含$\frac{kbd}{n}$ 个样本。因此，我们在专家的批大小上实现了 d因子的改进。

在分层专家混合（Section B）的情况下，主要的门控网络采用数据并行，而次级专家混合采用模型并行。每个次级专家混合驻留在一台设备上。

该技术使我们能够通过按比例增加训练集群中的设备数量来提升专家数量（从而提升参数数量）。总体批大小增加，同时保持每个专家的批大小不变。每台设备的内存和带宽需求也保持不变，步骤时间亦然，处理数量等于模型参数数量的训练样本所需的时间也同样保持不变。我们的目标是在一个一万亿词的语料库上训练一个一万亿参数的模型。截至撰写本文时，我们尚未将系统缩放到这一程度，但通过增加更多硬件应当可以实现。

利用卷积性： 在我们的语言模型中，我们对上一层的每个时间步应用同一个专家混合。如果我们等待上一层计算结束，就可以把所有时间步合并为一个大型批次来应用专家混合。这样做会按展开的时间步数的因子增大 MoE层的输入批大小。

为循环式专家混合增大批大小： 我们怀疑更强大的模型可能涉及以循环方式应用专家混合。例如，LSTM 或其他 RNN 的权重矩阵可以由专家混合替代。遗憾的是，这样的模型会破坏上一段中的卷积技巧，因为某一时间步的专家混合的输入依赖于前一时间步的专家混合的输出。Gruslys 等人（2016年）描述了一种技术，可在展开的 RNN 中大幅减少存储的激活数量，代价是重新计算前向传播激活。这将允许批大小大幅提升。

3.2 网络带宽

在分布式计算中，另一个主要的性能关注点是网络带宽。由于专家是固定的（见上文），且门控参数数量很小，大部分通信都涉及通过网络发送专家的输入和输出。为保持计算效率，单个专家的计算量与其输入和输出大小之比必须大于计算设备的计算与网络容量之比。对于 GPU，这个比值可能达到数千比一。在我们的实验中，我们使用带有一个隐藏层的专家，该层包含数千个 ReLU 激活单元。由于该专家中的权重矩阵大小为 $input_size \times hidden_size$和 $hidden\_size \times output\_size$，则计算与输入和输出之比等于隐藏层的大小。方便的是，我们只需使用更大的隐藏层，或更多隐藏层，即可提高计算效率。

4 平衡专家利用率

我们观察到，门控网络倾向于收敛到这样一种状态：它总是为同样的少量专家产生较大的权重。这种不平衡具有自我强化性，因为被偏爱的专家训练得更快，从而更容易被门控网络选择。Eigen 等人（2013年）描述了相同的现象，并在训练开始时使用硬约束以避免这个局部极小值。Bengio 等人（2015年）在每个门控的按批次平均值上加入了软约束。[1]

我们采用一种软约束方法。我们将某位专家相对于一个训练批次的重要性定义为该专家在该批次上的门控值之和。我们定义了一个额外的损失 $L_{importance}$，并将其加入到模型的整体损失函数中。该损失等于重要性值集合的变异系数的平方，并乘以一个手工调节的缩放因子 $w_{importance}$。这个额外的损失鼓励所有专家具有相等的重要性。

$$Importance(X) = \sum_{x \in X} G(x) \tag{6}$$

$$L_{importance}(X) = w_{importance} \cdot CV(Importance(X))^2 \tag{7}$$

[1]Bengio 等人（2015年）还引入了另外两个损失项。其中一个控制逐样本稀疏性，我们不需要它，因为这已由 k 的固定数值所约束。第三个损失项鼓励门控值的多样性。在我们的实验中，我们发现，随着专家的专门化（形成良性循环），门控值会自然呈现多样化，因此我们无需强制门控值的多样性。

尽管该损失函数可以确保重要性相等，专家仍可能接收到数量差异很大的样本。例如，一个专家可能接收少量但权重较大的样本，而另一个则接收大量但权重较小的样本。这会在分布式硬件上导致内存和性能问题。为了解决这个问题，我们引入第二个损失函数,L_{load}，用于确保负载均衡。附录A给出了该函数的定义以及实验结果。

5 实验

5.1 十亿词语言建模基准

Dat数据集： 该数据集由 (Chelba et al., 2013年) 引入，包含经过随机打乱的不重复句子 s fro来自新闻文章，总计约 8.29 亿词，词汇表大小为 793,471 个词 .

先前的当前最先进水平： 先前发表的最佳结果 (Jozefowicz et al., 2016年) 使用由一个或多个堆叠的 Long Short-Term Memory (LSTM) 层 (Hochreiter & Schmidhuber, 1997; Gers et al., 2000) 组成的模型。这些模型的 LSTM 层中的参数数量从 200 万到 1.51 亿不等。随着参数数量增加，质量大幅提升，计算成本也随之提高。这些模型的结果构成了图2右侧的上方曲线。

MoE 模型： 我们的模型由两个堆叠的 LSTM 层组成，它们之间夹有一个 MoE层（见图1）。我们改变了层的大小和专家数量。有关模型架构、训练方案、其他基线方法和结果的完整细节，见附录 C。

低计算量，容量可变： 为研究增加容量的效果，我们训练了一系列 MoE 模型，它们的计算成本大致相等：在前向传播中（不包含 Softmax 层），每个时间步每个训练样本约进行 800 万次乘加运算。我们将该指标称为（ops/timestep）。我们训练了扁平的 MoE，包含 4、32 和 256 个专家；以及分层的 MoE，包含 256、1024 和 4096 个专家。每个专家约有 100 万个参数。对于所有 MoE层，每个输入有 4 个专家处于激活状态。
这些模型的结果展示在图2 左侧。具有 4 个始终激活专家的模型（并不意外地）与计算成本匹配的基线模型表现相近，而其中最大的模型（4096 个专家）在测试集上的困惑度降低了令人印象深刻的 24%。

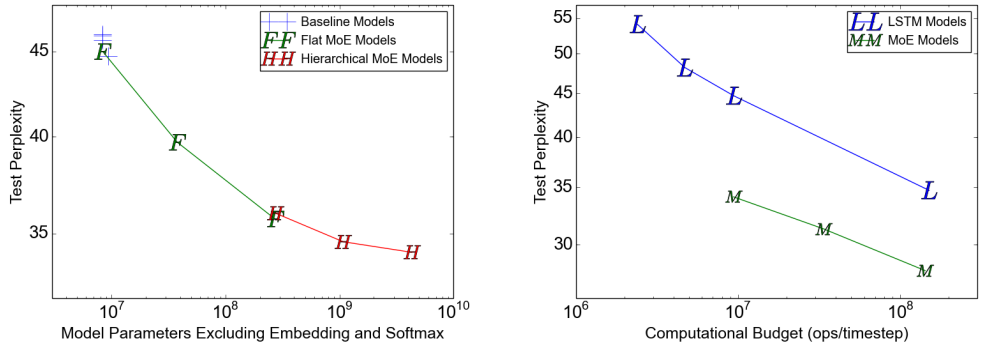

图2：在十亿词语言建模基准上的模型比较。左图：对于计算预算相近（每个时间步约800万次操作）的模型，绘制测试困惑度随模型容量变化的情况。右图：绘制测试困惑度随计算预算变化的情况。上方曲线代表（Jozefowicz 等，2016年）的 LSTM 模型。下方曲线代表在不同计算预算下的40亿参数的 MoE 模型。

多样化的计算，高容量： 除了上一节中的最大模型之外，我们又训练了两个具有相近高容量（40 亿个参数）但计算预算更高的 MoE 模型。这些模型采用更大的 LSTM，且专家数量更少但规模更大。详细信息

表1：在不同计算预算下高容量专家混合增强模型的汇总，与此前已发表的最佳结果（Jozefowicz 等，2016年）对比。详情见附录 C。

	Test 困惑度 10 轮	Test 困惑度 100轮	#参数 不含嵌入的 和 Softmax 层	操作数/时间步	训练 Time 10 轮	TFLOPS /GPU
已发表的最佳结果	34.7	30.6	1.51 亿	1.51 亿	59 小时，32 个 k40s	1.09
低预算 专家混合 模型	34.1		43.03 亿	890万	15 小时，16 k40s	0.74
中等预算的专家混合模型	31.3		43.13亿	3380万	17 小时，32 k40s	1.22
高预算的专家混合模型	**28.0**		4371 百万	142.7 百万	47小时，32 k40s	**1.56**

可在附录 C.2 中找到。这三个模型的结果构成了图2 右侧的最底线。表1将这些模型的结果与该数据集上此前已发表的最佳结果进行了比较。即便在控制训练轮数一致的条件下，其中最快的模型也超越了此前的最佳已发表结果，同时仅需 6% 的计算。

计算效率： 我们使用 TensorFlow (Abadi 等, 2016 年) 在包含 16-32 块 Tesla K40 GPU 的集群上训练我们的模型。对于每个模型，我们通过将处理一个训练批次所需的浮点运算次数除以观测到的步骤时间和集群中的 GPU 数量，来以 TFLOPs/GPU 确定计算效率。这里使用的运算计数高于我们在 ops/timestep 数值中报告的计数，因为我们包含了反向传播、包含了基于重要性采样的 Softmax 层训练，并且将一次乘加计为两个独立的运算。对于我们所有的 MoE 模型，专家所涉及的浮点运算占总量的介于三十七%到46%之间。

对于我们的基线模型（无专家混合），观测到的计算效率范围为 1.07-1.29 TFLOPs/GPU。对于我们的低计算量 MoE 模型，计算效率范围为 0.74- 0.90 TFLOPs/GPU，但 4 专家模型未能充分利用可用的并行。我们的最高计算量 MoE 模型在 1.56 TFLOPs/GPU 时更高效，这可能是因为矩阵更大。这些数值占英伟达声称的 4.29 TFLOPs/GPU 理论最大值的显著比例。详细结果见附录 C，表7。

5.2 1000 亿词 GOOGLE 新闻语料库

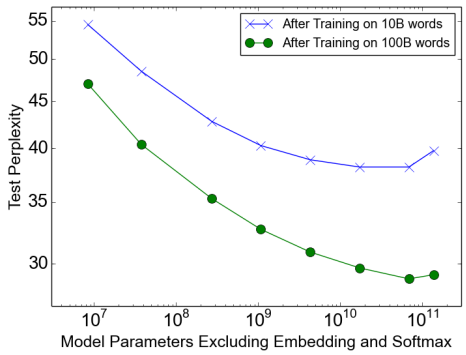

图3：在 1000 亿词语料库上的语言建模。模型的计算预算相近（每个时间步 800 万次操作）。

在 10 亿词语料库上，当 MoE层中的参数数量超过 10 亿时，增加额外的容量似乎产生了递减收益，如图2左侧所示。我们假设，对于更大的训练集，更高的容量将带来显著的质量提升。

我们构建了一个类似的训练集，由 Google 内部新闻语料库中去重并打乱的句子组成，总计约 1000 亿词。与上一节类似，我们测试了一系列具有相似计算成本的模型，约为每个时间步 800 万 ops/timestep。除了一个基线 LSTM 模型之外，我们还训练了加入 MoE层 的模型，其中包含 32, 256, 1024,

4096, 16384, 65536, 和 131072 个专家。这相当于 MoE层 中最多 1370 亿个参数。有关架构、训练和结果的详细信息见附录 D。

结果： 图3展示了在训练了100亿词（上方曲线）和1000亿词（下方曲线）之后，测试困惑度随容量变化的情况。对于完整的1000亿词训练，测试困惑度在专家数增加到65536（680亿参数）之前显著改善，相比计算匹配的基线下降了39%，但在131072个专家时出现了劣化，可能是由于稀疏性过高所致。两条曲线之间不断拉大的差距表明（并不意外）提高模型容量在更大的训练集上带来更大的收益。
即使在65536个专家（99.994% 层稀疏性）的设置下，模型的计算效率仍达到可观的 0.72 TFLOPS/GPU。

5.3 机器翻译（单语言对）

模型架构： 我们的模型是（Wu et al., 2016年）中所述 GNMT 模型的修改版本。为了减少计算，我们将编码器和解码器中的 LSTM 层数分别从 9 和 8 降至 3 和 2。我们在编码器（第2层与第3层之间）和解码器（第1层与第2层之间）都插入了 MoE层。每个 MoE层包含多达 2048个专家，每个约有两百万参数，累计为模型增加了约 80亿参数。有关模型架构、测试流程和结果的更多细节见附录E。

数据集： 我们在 WMT'14 En→Fr 和 En→De 语料库上对我们的方法进行了基准评测，其训练集分别包含 3600 万对句子和 500 万对句子。实验流程也与 (Wu et al., 2016年) 中的类似：使用 newstest2014 作为测试集，与以往工作 (Luong et al., 2015a; Zhou et al., 2016年; Wu et al., 2016年) 进行比较，而 newstest2012 与 newstest2013 的组合被用作开发集。我们还在 Google 的 Production English to French 数据上测试了相同的模型。

表2：在 WMT'14 En→Fr newstest2014 上的结果（粗体值表示最佳结果）。

模型	Test 困惑度	测试 BLEU	操作数/时间步	总计 #参数	训练 Time
具有2048个专家的专家混合	2.69	40.35	85M	8.7B	3天/64 k40s
具有2048个专家的专家混合（更长的训练）	**2.63**	**40.56**	85M	8.7B	6天/64 k40s
GNMT（Wu 等，2016年）	2.79	39.22	214M	278M	6 天/96 k80s
GNMT+RL（Wu 等，2016年）	2.96	39.92	214M	278M	6 天/96 k80s
PBMT（Durrani 等，2014年）		37.0			
LSTM（6 层）（Luong 等，2015年b）		31.5			
LSTM（6 层+PosUnk）（Luong 等，2015年b）		33.1			
DeepAtt (Zhou et al., 2016年)		37.7			
DeepAtt+PosUnk (Zhou et al., 2016年)		39.2			

表3：在 WMT'14 En → De newstest2014 上的结果（粗体值表示最佳结果）。

模型	Test 困惑度	Test BLEU	操作数/时间步	总计 #参数	训练 Time
具有2048个专家的专家混合	**4.64**	**26.03**	85M	8.7B	1天/64 k40s
GNMT (Wu 等，2016年)	5.25	24.91	214M	278M	1天/96 k80s
GNMT +RL（Wu 等，2016年）	8.08	24.66	214M	278M	1天/96 k80s
PBMT（Durrani 等，2014年）		20.7			
DeepAtt（Zhou 等，2016年）		20.6			

表4：在 Google 生产环境 En→ Fr 数据集上的结果（加粗的值表示最佳结果).

模型	Eval 困惑度	Eval BLEU	Test 困惑度	Test BLEU	ops/timestep	总计 #参数	训练 Time
具有2048个专家的专家混合	**2.60**	**37.27**	**2.69**	**36.57**	85M	8.7B	1 天/64 k40s
GNMT（Wu 等，2016年）	2.78	35.80	2.87	35.56	214M	278M	6 天/96 k80s

结果： 表2、表3和表4展示了我们最大规模模型的结果，并与已发表的结果进行比较。我们的方法在 WMT'14 En→Fr 和 En→De 基准上取得了 40.56 和 26.03 的 BLEU 分数。由于我们的模型未使用 RL 精炼，这些结果在 (Wu et al., 2016年) 所述强大基线方法之上分别带来了 1.34 和 1.12 的 BLEU 提升。困惑度得分也更好。[2] 在 Google Production 数据集上，即使仅训练了六分之一的时间，我们的模型在测试 BLEU 上仍高出 1.01。

5.4 多语言机器翻译

数据集： （Johnson 等，2016年）在包含十二个语言对的一个非常大的合并数据集上训练了一个 GNMT（Wu 等，2016年）模型。其结果略逊于对 12 个分别训练的单对 GNMT 模型的结果。这并不令人惊讶，因为那十二个模型的容量以及累计训练都相当于该单个模型的 12 倍。我们用一个加入专家混合的单个模型重复了该实验。有关模型架构的详细信息见附录E。我们在与（Johnson 等，2016年）相同的数据集上训练我们的模型，并处理了相同数量的训练样本（约 30 亿个句对）。由于我们的模型计算预算更低，我们的训练时间更短。

结果： 单对 GNMT 模型、多语言 GNMT 模型以及多语言专家混合模型的结果见表5。专家混合模型在开发集上的困惑度比多语言 GNMT 模型低 19%。在 BLEU 分数上，专家混合模型在 12 个语言对中的 11 个上显著优于多语言 GNMT 模型（最多高出 5.84 分），并且在 12 个语言对中的 8 个上甚至优于单语 GNMT 模型。在 English→ Korean 上的较差性能似乎是严重过度训练的结果，因为对于较稀有的语言对，训练语料中的少量真实样本被高度过采样。

表5：多语言机器翻译（加粗的数值表示最佳结果）。

	GNMT-Mono	GNMT-Multi	MoE-Multi	MoE-Multi 对 GNMT-Multi
参数 278M / 模型		278M	8.7B	
操作/时间步	212M	212M	102M	
训练时间，硬件	多种	21 天，96 k20s	**12 天，64 k40s**	
困惑度（开发集）		4.14	**3.35**	-19%
法语 → 英语 测试 BLEU	36.47	34.40	**37.46**	+3.06
德语 → 英语 测试 BLEU	31.77	31.17	**34.80**	+3.63
日语 → 英语 测试 BLEU	23.41	21.62	**25.91**	+4.29
韩语 → 英语 测试 BLEU	25.42	22.87	**28.71**	+5.84
Portuguese → English 测试 BLEU	44.40	42.53	**46.13**	+3.60
Spanish → English 测试 BLEU	38.00	36.04	**39.39**	+3.35
English → French 测试 BLEU	35.37	34.00	**36.59**	+2.59
英语 → 德语 测试集 BLEU	**26.43**	23.15	24.53	+1.38
英语 → 日语 测试集 BLEU	**23.66**	21.10	22.78	+1.68
英语 →韩语 测试集 BLEU	**19.75**	18.41	16.62	-1.79
英语 → 葡萄牙语 测试 BLEU	**38.40**	37.35	37.90	+0.55
英语 → 西班牙语 测试 BLEU	34.50	34.25	**36.21**	+1.96

6 结论

本工作首次展示了在深度网络中通过条件计算取得的显著收益。我们认真梳理了条件计算的设计考量与挑战，并通过结合算法与工程方案加以解决。尽管我们聚焦于文本，只要拥有足够大的训练集，条件计算同样可能在其他领域发挥作用。我们期待在未来几年看到大量关于条件计算的全新实现与应用。

致谢

我们谨向在本项目中给予我们帮助的 Google Brain 和 Google Translate 团队的所有成员表示感谢，尤其是 Zhifeng Chen、Yonghui Wu 和 Melvin Johnson。同时也感谢匿名的 ICLR 审稿人提出的宝贵建议，帮助我们改进这篇论文。

[2]报告的困惑度是相对于我们模型和 GNMT 均使用的标记化方案。

REFERENCES

Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Gregory S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian J. Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Józefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dan Mané, Rajat Monga, Sherry Moore, Derek Gordon Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul A. Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda B. Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. Tensorflow: Large-scale machine learning on heterogeneous distributed systems. *CoRR*, abs/1603.04467, 2016. URL `http://arxiv.org/abs/1603.04467`.

Rahaf Aljundi, Punarjay Chakravarty, and Tinne Tuytelaars. Expert gate: Lifelong learning with a network of experts. *CoRR*, abs/1611.06194, 2016. URL `http://arxiv.org/abs/1611.06194`.

A. Almahairi, N. Ballas, T. Cooijmans, Y. Zheng, H. Larochelle, and A. Courville. Dynamic Capacity Networks. *ArXiv e-prints*, November 2015.

Dario Amodei, Rishita Anubhai, Eric Battenberg, Carl Case, Jared Casper, Bryan Catanzaro, Jingdong Chen, Mike Chrzanowski, Adam Coates, Greg Diamos, Erich Elsen, Jesse Engel, Linxi Fan, Christopher Fougner, Tony Han, Awni Y. Hannun, Billy Jun, Patrick LeGresley, Libby Lin, Sharan Narang, Andrew Y. Ng, Sherjil Ozair, Ryan Prenger, Jonathan Raiman, Sanjeev Satheesh, David Seetapun, Shubho Sengupta, Yi Wang, Zhiqian Wang, Chong Wang, Bo Xiao, Dani Yogatama, Jun Zhan, and Zhenyao Zhu. Deep speech 2: End-to-end speech recognition in english and mandarin. *arXiv preprint arXiv:1512.02595*, 2015.

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.

Emmanuel Bengio, Pierre-Luc Bacon, Joelle Pineau, and Doina Precup. Conditional computation in neural networks for faster models. *arXiv preprint arXiv:1511.06297*, 2015.

Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

Ciprian Chelba, Tomas Mikolov, Mike Schuster, Qi Ge, Thorsten Brants, Phillipp Koehn, and Tony Robinson. One billion word benchmark for measuring progress in statistical language modeling. *arXiv preprint arXiv:1312.3005*, 2013.

K. Cho and Y. Bengio. Exponentially Increasing the Capacity-to-Computation Ratio for Conditional Computation in Deep Learning. *ArXiv e-prints*, June 2014.

Ronan Collobert, Samy Bengio, and Yoshua Bengio. A parallel mixture of SVMs for very large scale problems. *Neural Computing*, 2002.

Andrew Davis and Itamar Arel. Low-rank approximations for conditional feedforward computation in deep neural networks. *arXiv preprint arXiv:1312.4461*, 2013.

Marc Peter Deisenroth and Jun Wei Ng. Distributed Gaussian processes. In *ICML*, 2015.

John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization, 2010.

Nadir Durrani, Barry Haddow, Philipp Koehn, and Kenneth Heafield. Edinburgh's phrase-based machine translation systems for wmt-14. In *Proceedings of the Ninth Workshop on Statistical Machine Translation*, 2014.

David Eigen, Marc'Aurelio Ranzato, and Ilya Sutskever. Learning factored representations in a deep mixture of experts. *arXiv preprint arXiv:1312.4314*, 2013.

Ekaterina Garmash and Christof Monz. Ensemble learning for multi-source neural machine translation. In *staff.science.uva.nl/c.monz*, 2016.

Felix A. Gers, Jürgen A. Schmidhuber, and Fred A. Cummins. Learning to forget: Continual prediction with lstm. *Neural Computation*, 2000.

Audrunas Gruslys, Rémi Munos, Ivo Danihelka, Marc Lanctot, and Alex Graves. Memory-efficient backpropagation through time. *CoRR*, abs/1606.03401, 2016. URL `http://arxiv.org/abs/1606.03401`.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *IEEE Conference on Computer Vision and Pattern Recognition*, 2015.

Geoffrey Hinton, Li Deng, Dong Yu, George E. Dahl, Abdel-rahman Mohamed, Navdeep Jaitly, Andrew Senior, Vincent Vanhoucke, Patrick Nguyen, Tara N. Sainath, et al. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE Signal Processing Magazine*, 2012.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 1997.

Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015.

Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computing*, 1991.

Melvin Johnson, Mike Schuster, Quoc V. Le, Maxim Krikun, Yonghui Wu, Zhifeng Chen, Nikhil Thorat, Fernanda B. Viégas, Martin Wattenberg, Greg Corrado, Macduff Hughes, and Jeffrey Dean. Google's multilingual neural machine translation system: Enabling zero-shot translation. *CoRR*, abs/1611.04558, 2016. URL `http://arxiv.org/abs/1611.04558`.

Michael I. Jordan and Robert A. Jacobs. Hierarchical mixtures of experts and the EM algorithm. *Neural Computing*, 1994.

Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*, 2016.

Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.

Reinhard Kneser and Hermann. Ney. Improved backingoff for m-gram language modeling., 1995.

Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *NIPS*, 2012.

Quoc V. Le, Marc'Aurelio Ranzato, Rajat Monga, Matthieu Devin, Kai Chen, Greg S. Corrado, Jeffrey Dean, and Andrew Y. Ng. Building high-level features using large scale unsupervised learning. In *ICML*, 2012.

Patrick Gallinari Ludovic Denoyer. Deep sequential neural network. *arXiv preprint arXiv:1410.0510*, 2014.

Minh-Thang Luong, Hieu Pham, and Christopher D. Manning. Effective approaches to attention-based neural machine translation. *EMNLP*, 2015a.

Minh-Thang Luong, Ilya Sutskever, Quoc V. Le, Oriol Vinyals, and Wojciech Zaremba. Addressing the rare word problem in neural machine translation. *ACL*, 2015b.

Carl Edward Rasmussen and Zoubin Ghahramani. Infinite mixtures of Gaussian process experts. *NIPS*, 2002.

Hasim Sak, Andrew W Senior, and Françoise Beaufays. Long short-term memory recurrent neural network architectures for large scale acoustic modeling. In *INTERSPEECH*, pp. 338–342, 2014.

Mike Schuster and Kaisuke Nakajima. Japanese and Korean voice search. *ICASSP*, 2012.

Babak Shahbaba and Radford Neal. Nonlinear models using dirichlet process mixtures. *JMLR*, 2009.

Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *NIPS*, 2014.

Lucas Theis and Matthias Bethge. Generative image modeling using spatial LSTMs. In *NIPS*, 2015.

Volker Tresp. Mixtures of Gaussian Processes. In *NIPS*, 2001.

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Łukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

Bangpeng Yao, Dirk Walther, Diane Beck, and Li Fei-fei. Hierarchical mixture of classification experts uncovers interactions between brain regions. In *NIPS*. 2009.

Wojciech Zaremba, Ilya Sutskever, and Oriol Vinyals. Recurrent neural network regularization. *arXiv preprint arXiv:1409.2329*, 2014.

Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. *arXiv preprint arXiv:1606.04199*, 2016.

APPENDICES

A LOAD-BALANCING LOSS

As discussed in section 4, for load-balancing purposes, we want to define an additional loss function to encourage experts to receive roughly equal numbers of training examples. Unfortunately, the number of examples received by an expert is a discrete quantity, so it can not be used in back-propagation. Instead, we define a smooth estimator $Load(X)$ of the number of examples assigned to each expert for a batch X of inputs. The smoothness allows us to back-propagate gradients through the estimator. This is the purpose of the noise term in the gating function. We define $P(x,i)$ as the probability that $G(x)_i$ is nonzero, given a new random choice of noise on element i, but keeping the already-sampled choices of noise on the other elements. To compute $P(x,i)$, we note that the $G(x)_i$ is nonzero if and only if $H(x)_i$ is greater than the k^{th}-greatest element of $H(x)$ excluding itself. The probability works out to be:

$$P(x,i) = Pr\Big((x \cdot W_g)_i + StandardNormal() \cdot Softplus((x \cdot W_{noise})_i) \\ > kth\_excluding(H(x),k,i)\Big) \tag{8}$$

Where $kth\_excluding(v,k,i)$ means the kth highest component of v, excluding component i. Simplifying, we get:

$$P(x,i) = \Phi\Big(\frac{(x \cdot W_g)_i - kth\_excluding(H(x),k,i)}{Softplus((x \cdot W_{noise})_i)}\Big) \tag{9}$$

Where Φ is the CDF of the standard normal distribution.

$$Load(X)_i = \sum_{x \in X} P(x,i) \tag{10}$$

We can now define the load loss to be the square of the coefficient of variation of the load vector, multiplied by a hand-tuned scaling factor w_{load}.

$$L_{load}(X) = w_{load} \cdot CV(Load(X))^2 \tag{11}$$

Initial Load Imbalance: To avoid out-of-memory errors, we need to initialize the network in a state of approximately equal expert load (since the soft constraints need some time to work). To accomplish this, we initialize the matrices W_g and W_{noise} to all zeros, which yields no signal and some noise.

Experiments: We trained a set of models with identical architecture (the MoE-256 model described in Appendix C), using different values of $w_{importance}$ and w_{load}. We trained each model for 10 epochs, then measured perplexity on the test set. We also measured the coefficients of variation in $Importance$ and $Load$, as well as ratio of the load on the most overloaded expert to the average load. This last value is significant for load balancing purposes on distributed hardware. All of these metrics were averaged over several training batches.

Table 6: Experiments with different combinations of losses.

$w_{importance}$	w_{load}	Test Perplexity	$CV(Importance(X))$	$CV(Load(X))$	$\frac{max(Load(X))}{mean(Load(X))}$
0.0	0.0	39.8	3.04	3.01	17.80
0.2	0.0	**35.6**	0.06	0.17	1.47
0.0	0.2	35.7	0.22	0.04	1.15
0.1	0.1	**35.6**	0.06	0.05	1.14
0.01	0.01	35.7	0.48	0.11	1.37
1.0	1.0	35.7	0.03	0.02	**1.07**

Results: Results are reported in Table 6. All the combinations containing at least one the two losses led to very similar model quality, where having no loss was much worse. Models with higher values of w_{load} had lower loads on the most overloaded expert.

B HIERACHICAL MIXTURE OF EXPERTS

If the number of experts is very large, we can reduce the branching factor by using a two-level hierarchical MoE. In a hierarchical MoE, a primary gating network chooses a sparse weighted combination of "experts", each of which is itself a secondary mixture-of-experts with its own gating network.[3] If the hierarchical MoE consists of a groups of b experts each, we denote the primary gating network by $G_{primary}$, the secondary gating networks by $(G_1, G_2..G_a)$, and the expert networks by $(E_{0,0}, E_{0,1}..E_{a,b})$. The output of the MoE is given by:

$$y_H = \sum_{i=1}^{a}\sum_{j=1}^{b} G_{primary}(x)_i \cdot G_i(x)_j \cdot E_{i,j}(x) \tag{12}$$

Our metrics of expert utilization change to the following:

$$Importance_H(X)_{i,j} = \sum_{x \in X} G_{primary}(x)_i \cdot G_i(x)_j \tag{13}$$

$$Load_H(X)_{i,j} = \frac{Load_{primary}(X)_i \cdot Load_i(X^{(i)})_j}{|X^{(i)}|} \tag{14}$$

$Load_{primary}$ and $Load_i$ deonte the $Load$ functions for the primary gating network and i^{th} secondary gating network respectively. $X^{(i)}$ denotes the subset of X for which $G_{primary}(x)_i > 0$.

It would seem simpler to let $Load_H(X)_{i,j} = Load_i(X_i)_j$, but this would not have a gradient with respect to the primary gating network, so we use the formulation above.

C 1 BILLION WORD LANGUAGE MODELING BENCHMARK - EXPERIMENTAL DETAILS

C.1 8-MILLION-OPERATIONS-PER-TIMESTEP MODELS

Model Architecture: Our model consists of five layers: a word embedding layer, a recurrent Long Short-Term Memory (LSTM) layer (Hochreiter & Schmidhuber, 1997; Gers et al., 2000), a MoE layer, a second LSTM layer, and a softmax layer. The dimensionality of the embedding layer, the number of units in each LSTM layer, and the input and output dimensionality of the MoE layer are all equal to 512. For every layer other than the softmax, we apply drouput (Zaremba et al., 2014) to the layer output, dropping each activation with probability $DropProb$, otherwise dividing by $(1 - DropProb)$. After dropout, the output of the previous layer is added to the layer output. This residual connection encourages gradient flow (He et al., 2015).

MoE Layer Architecture: Each expert in the MoE layer is a feed forward network with one ReLU-activated hidden layer of size 1024 and an output layer of size 512. Thus, each expert contains $[512 * 1024] + [1024 * 512] = 1M$ parameters. The output of the MoE layer is passed through a sigmoid function before dropout. We varied the number of experts between models, using ordinary MoE layers with 4, 32 and 256 experts and hierarchical MoE layers with 256, 1024 and 4096 experts. We call the resulting models MoE-4, MoE-32, MoE-256, MoE-256-h, MoE-1024-h and MoE-4096-h. For the hierarchical MoE layers, the first level branching factor was 16, corresponding to the number of GPUs in our cluster. We use Noisy-Top-K Gating (see Section 2.1) with $k = 4$ for the ordinary MoE layers and $k = 2$ at each level of the hierarchical MoE layers. Thus, each example is processed by exactly 4 experts for a total of 4M ops/timestep. The two LSTM layers contribute 2M ops/timestep each for the desired total of 8M.

[3] We have not found the need for deeper hierarchies.

Computationally-Matched Baselines: The MoE-4 model does not employ sparsity, since all 4 experts are always used. In addition, we trained four more computationally-matched baseline models with no sparsity:

- MoE-1-Wide: The MoE layer consists of a single "expert" containing one ReLU-activated hidden layer of size 4096.
- MoE-1-Deep: The MoE layer consists of a single "expert" containing four ReLU-activated hidden layers, each with size 1024.
- 4xLSTM-512: We replace the MoE layer with two additional 512-unit LSTM layers.
- LSTM-2048-512: The model contains one 2048-unit LSTM layer (and no MoE). The output of the LSTM is projected down to 512 dimensions (Sak et al., 2014). The next timestep of the LSTM receives the projected output. This is identical to one of the models published in (Jozefowicz et al., 2016). We re-ran it to account for differences in training regimen, and obtained results very similar to the published ones.

Training: The models were trained on a cluster of 16 K40 GPUs using the synchronous method described in Section 3. Each batch consisted of a set of sentences totaling roughly 300,000 words. In the interest of time, we limited training to 10 epochs, (27,000 steps). Training took 12-16 hours for all models, except for MoE-4, which took 18 hours (since all the expert computation was performed on only 4 of 16 GPUs). We used the Adam optimizer (Kingma & Ba, 2015). The base learning rate was increased linearly for the first 1000 training steps, and decreased after that so as to be proportional to the inverse square root of the step number. The Softmax output layer was trained efficiently using importance sampling similarly to the models in (Jozefowicz et al., 2016). For each model, we performed a hyper-parmeter search to find the best dropout probability, in increments of 0.1.

To ensure balanced expert utilization we set $w_{importance} = 0.1$ and $w_{load} = 0.1$, as described in Section 4 and Appendix A.

Results: We evaluate our model using perplexity on the holdout dataset, used by (Chelba et al., 2013; Jozefowicz et al., 2016). We follow the standard procedure and sum over all the words including the end of sentence symbol. Results are reported in Table 7. For each model, we report the test perplexity, the computational budget, the parameter counts, the value of $DropProb$, and the computational efficiency.

Table 7: Model comparison on 1 Billion Word Language Modeling Benchmark. Models marked with * are from (Jozefowicz et al., 2016).

Model	Test Perplexity 10 epochs	Test Perplexity (final)	ops/timestep (millions)	#Params excluding embed. & softmax (millions)	Total #Params (billions)	$Drop-Prob$	TFLOPS per GPU (observed)
Kneser-Ney 5-gram*		67.6	0.00001		1.8		
LSTM-512-512*		54.1	2.4	2.4	0.8	0.1	
LSTM-1024-512*		48.2	4.7	4.7	0.8	0.1	
LSTM-2048-512*	45.0	43.7	9.4	9.4	0.8	0.1	0.61
LSTM-2048-512	44.7		9.4	9.4	0.8	0.1	1.21
4xLSTM-512	46.0		8.4	8.4	0.8	0.1	1.07
MoE-1-Wide	46.1		8.4	8.4	0.8	0.1	1.29
MoE-1-Deep	45.7		8.4	8.4	0.8	0.1	1.29
MoE-4	45.0		8.4	8.4	0.8	0.1	0.52
MoE-32	39.7		8.4	37.8	0.9	0.1	0.87
MoE-256	35.7		8.6	272.9	1.1	0.1	0.81
MoE-256-h	36.0		8.4	272.9	1.1	0.1	0.89
MoE-1024-h	34.6		8.5	1079.0	1.9	0.2	0.90
MoE-4096-h	34.1		8.9	4303.4	5.1	0.2	0.74
2xLSTM-8192-1024*	34.7	30.6	151.0	151.0	1.8	0.25	1.09
MoE-34M	31.3		33.8	4313.9	6.0	0.3	1.22
MoE-143M	**28.0**		142.7	4371.1	6.0	0.4	**1.56**

C.2 MORE EXPENSIVE MODELS

We ran two additional models (MoE-34M and MoE-143M) to investigate the effects of adding more computation in the presence of a large MoE layer. These models have computation budgets of 34M and 143M ops/timestep. Similar to the models above, these models use a MoE layer between two LSTM layers. The dimensionality of the embedding layer, and the input and output dimensionality of the MoE layer are set to 1024 instead of 512. For MoE-34M, the LSTM layers have 1024 units. For MoE-143M, the LSTM layers have 4096 units and an output projection of size 1024 (Sak et al., 2014). MoE-34M uses a hierarchical MoE layer with 1024 experts, each with a hidden layer of size 2048. MoE-143M uses a hierarchical MoE layer with 256 experts, each with a hidden layer of size 8192. Both models have 4B parameters in the MoE layers. We searched for the best $DropProb$ for each model, and trained each model for 10 epochs.

The two models achieved test perplexity of 31.3 and 28.0 respectively, showing that even in the presence of a large MoE, more computation is still useful. Results are reported at the bottom of Table 7. The larger of the two models has a similar computational budget to the best published model from the literature, and training times are similar. Comparing after 10 epochs, our model has a lower test perplexity by 18%.

D 100 BILLION WORD GOOGLE NEWS CORPUS - EXPERIMENTAL DETAILS

Model Architecture: The models are similar in structure to the 8-million-operations-per-timestep models described in the previous section. We vary the number of experts between models, using an ordinary MoE layer with 32 experts and hierarchical MoE layers with 256, 1024, 4096, 16384, 65536 and 131072 experts. For the hierarchical MoE layers, the first level branching factors are 32, 32, 64, 128, 256 and 256, respectively.

Training: Models are trained on a cluster of 32 Tesla K40 GPUs, except for the last two models, which are trained on clusters of 64 and 128 GPUs so as to have enough memory for all the parameters. For all models, training batch sizes are approximately 2.5 million words. Models are trained once-through over about 100 billion words.

We implement several memory optimizations in order to fit up to 1 billion parameters per GPU. First, we do not store the activations of the hidden layers of the experts, but instead recompute them on the backwards pass. Secondly, we modify the optimizer on the expert parameters to require less auxiliary storage:

The Adam optimizer (Kingma & Ba, 2015) keeps first and second moment estimates of the per-parameter gradients. This triples the required memory. To avoid keeping a first-moment estimator, we set $\beta_1 = 0$. To reduce the size of the second moment estimator, we replace it with a factored approximation. For a matrix of parameters, instead of maintaining a full matrix of second-moment estimators, we maintain vectors of row-wise and column-wise averages of that matrix. At each step, the matrix of estimators is taken to be the outer product of those two vectors divided by the mean of either one. This technique could similarly be applied to Adagrad (Duchi et al., 2010).

Table 8: Model comparison on 100 Billion Word Google News Dataset

Model	Test Perplexity .1 epochs	Test Perplexity 1 epoch	ops/timestep (millions)	#Params excluding embed. & softmax (millions)	Total #Params (billions)	TFLOPS per GPU (observed)
Kneser-Ney 5-gram	67.1	45.3	0.00001		76.0	
4xLSTM-512	54.5	47.0	8.4	8.4	0.1	**1.23**
MoE-32	48.5	40.4	8.4	37.8	0.1	0.83
MoE-256-h	42.8	35.3	8.4	272.9	0.4	1.11
MoE-1024-h	40.3	32.7	8.5	1079.0	1.2	1.14
MoE-4096-h	38.9	30.9	8.6	4303.4	4.4	1.07
MoE-16384-h	**38.2**	29.7	8.8	17201.0	17.3	0.96
MoE-65536-h	**38.2**	**28.9**	9.2	68791.0	68.9	0.72
MoE-131072-h	39.8	29.2	9.7	137577.6	137.7	0.30

Results: We evaluate our model using perplexity on a holdout dataset. Results are reported in Table 8. Perplexity after 100 billion training words is 39% lower for the 68-billion-parameter MoE

model than for the baseline model. It is notable that the measured computational efficiency of the largest model (0.30 TFLOPS/GPU) is very low compared to the other models. This is likely a result of the fact that, for purposes of comparison to the other models, we did not increase the training batch size proportionally to the number of GPUs. For comparison, we include results for a computationally matched baseline model consisting of 4 LSTMs, and for an unpruned 5-gram model with Kneser-Ney smoothing (Kneser & Ney, 1995).[4]

E MACHINE TRANSLATION - EXPERIMENTAL DETAILS

Model Architecture for Single Language Pair MoE Models: Our model is a modified version of the GNMT model described in (Wu et al., 2016). To reduce computation, we decrease the number of LSTM layers in the encoder and decoder from 9 and 8 to 3 and 2 respectively. We insert MoE layers in both the encoder (between layers 2 and 3) and the decoder (between layers 1 and 2). We use an attention mechanism between the encoder and decoder, with the first decoder LSTM receiving output from and providing input for the attention [5]. All of the layers in our model have input and output dimensionality of 512. Our LSTM layers have 2048 hidden units, with a 512-dimensional output projection. We add residual connections around all LSTM and MoE layers to encourage gradient flow (He et al., 2015). Similar to GNMT, to effectively deal with rare words, we used sub-word units (also known as "wordpieces") (Schuster & Nakajima, 2012) for inputs and outputs in our system.

We use a shared source and target vocabulary of 32K wordpieces. We also used the same beam search technique as proposed in (Wu et al., 2016).

We train models with different numbers of experts in the MoE layers. In addition to a baseline model with no MoE layers, we train models with flat MoE layers containing 32 experts, and models with hierarchical MoE layers containing 512 and 2048 experts. The flat MoE layers use $k = 4$ and the hierarchical MoE models use $k = 2$ at each level of the gating network. Thus, each input is processed by exactly 4 experts in each MoE layer. Each expert in the MoE layer is a feed forward network with one hidden layer of size 2048 and ReLU activation. Thus, each expert contains $[512 * 2048] + [2048 * 512] = 2M$ parameters. The output of the MoE layer is passed through a sigmoid function. We use the strictly-balanced gating function described in Appendix F.

Model Architecture for Multilingual MoE Model: We used the same model architecture as for the single-language-pair models, with the following exceptions: We used noisy-top-k gating as described in Section 2.1, not the scheme from Appendix F. The MoE layers in the encoder and decoder are non-hierarchical MoEs with $n = 512$ experts, and $k = 2$. Each expert has a larger hidden layer of size 8192. This doubles the amount of computation in the MoE layers, raising the computational budget of the entire model from 85M to 102M ops/timestep.

Training: We trained our networks using the Adam optimizer (Kingma & Ba, 2015). The base learning rate was increased linearly for the first 2000 training steps, held constant for an additional 8000 steps, and decreased after that so as to be proportional to the inverse square root of the step number. For the single-language-pair models, similarly to (Wu et al., 2016), we applied dropout (Zaremba et al., 2014) to the output of all embedding, LSTM and MoE layers, using $DropProb = 0.4$. Training was done synchronously on a cluster of up to 64 GPUs as described in section 3. Each training batch consisted of a set of sentence pairs containing roughly 16000 words per GPU.

To ensure balanced expert utilization we set $w_{importance} = 0.01$ and $w_{load} = 0.01$, as described in Section 4 and Appendix A.

Metrics: We evaluated our models using the perplexity and the standard BLEU score metric. We reported tokenized BLEU score as computed by the multi-bleu.pl script, downloaded from the public implementation of Moses (on Github), which was also used in (Luong et al., 2015a).

[4] While the original size of the corpus was 130 billion words, the neural models were trained for a maximum of 100 billion words. The reported Kneser-Ney 5-gram models were trained over 13 billion and 130 billion words respectively, giving them a slight advantage over the other reported results.

[5] For performance reasons, we use a slightly different attention function from the one described in (Wu et al., 2016) - See Appendix G

Results: Tables 2, 3 and 4 in Section 5.3 show comparisons of our results to other published methods. Figure 4 shows test perplexity as a function of number of words in the (training data's) source sentences processed for models with different numbers of experts. As can be seen from the Figure, as we increased the number of experts to approach 2048, the test perplexity of our model continued to improve.

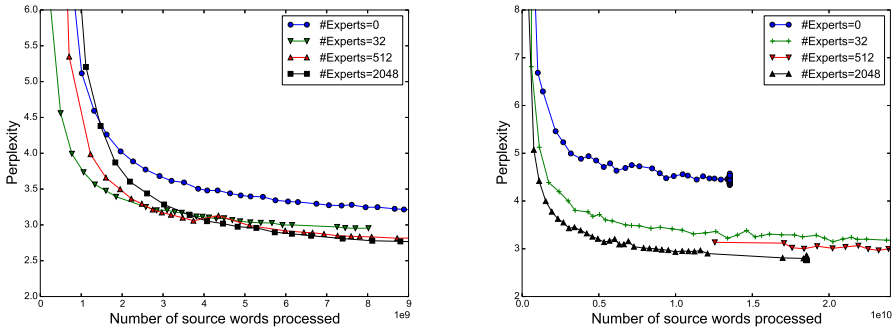

Figure 4: Perplexity on WMT'14 En→ Fr (left) and Google Production En→ Fr (right) datasets as a function of number of words processed. The large differences between models at the beginning of training are due to different batch sizes. All models incur the same computational budget (85M ops/timestep) except the one with no experts.

We found that the experts indeed become highly specialized by syntax and/or semantics, as can be seen in Table 9. For example, one expert is used when the indefinite article "a" introduces the direct object in a verb phrase indicating importance or leadership.

Table 9: Contexts corresponding to a few of the 2048 experts in the MoE layer in the encoder portion of the WMT'14 En→ Fr translation model. For each expert i, we sort the inputs in a training batch in decreasing order of $G(x)_i$, and show the words surrounding the corresponding positions in the input sentences.

Expert 381	Expert 752	Expert 2004
... with **researchers** , ...	... plays **a** core ...	... with **rapidly** growing ...
... to **innovation** .	... plays **a** critical ...	... under **static** conditions ...
... tics **researchers** .	... provides **a** legislative ...	... to **swift** ly ...
... the **generation** of ...	... play **a** leading ...	... to **dras** tically ...
... technology **innovations** is ...	... assume **a** leadership ...	... the **rapid** and ...
... technological **innovations** , ...	... plays **a** central ...	... the **fast** est ...
... support **innovation** throughout ...	... taken **a** leading ...	... the **Quick** Method ...
... role **innovation** will ...	... established **a** reconciliation ...	... rec **urrent**) ...
... research **scienti** st ...	... played **a** vital ...	... provides **quick** access ...
... promoting **innovation** where ...	... have **a** central ...	... of **volatile** organic ...
...	...	...

F STRICTLY BALANCED GATING

Due to some peculiarities in our infrastructure which have since been fixed, at the time we ran some of the machine translation experiments, our models ran faster if every expert received exactly the same batch size. To accommodate this, we used a different gating function which we describe below.

Recall that we define the softmax gating function to be:

$$G_\sigma(x) = Softmax(x \cdot W_g) \tag{15}$$

Sparse Gating (alternate formulation): To obtain a sparse gating vector, we multiply $G_\sigma(x)$ component-wise with a sparse mask $M(G_\sigma(x))$ and normalize the output. The mask itself is a function of $G_\sigma(x)$ and specifies which experts are assigned to each input example:

$$G(x)_i = \frac{G_\sigma(x)_i M(G_\sigma(x))_i}{\sum_{j=1}^{n} G_\sigma(x)_j M(G_\sigma(x))_j} \tag{16}$$

Top-K Mask: To implement top-k gating in this formulation, we would let $M(v) = TopK(v, k)$, where:

$$TopK(v, k)_i = \begin{cases} 1 & \text{if } v_i \text{ is in the top } k \text{ elements of } v. \\ 0 & \text{otherwise.} \end{cases} \tag{17}$$

Batchwise Mask: To force each expert to receive the exact same number of examples, we introduce an alternative mask function, $M_{batchwise}(X, m)$, which operates over batches of input vectors. Instead of keeping the top k values per example, we keep the top m values per expert across the training batch, where $m = \frac{k|X|}{n}$, so that each example is sent to an average of k experts.

$$M_{batchwise}(X, m)_{j,i} = \begin{cases} 1 & \text{if } X_{j,i} \text{ is in the top } m \text{ values for to expert } i \\ 0 & \text{otherwise} \end{cases} \tag{18}$$

As our experiments suggest and also observed in (Ioffe & Szegedy, 2015), using a batchwise function during training (such as $M_{batchwise}$) requires modifications to the inference when we may not have a large batch of examples. Our solution to this is to train a vector T of per-expert threshold values to approximate the effects of the batchwise mask. We use the following mask at inference time:

$$M_{threshold}(x, T)_i = \begin{cases} 1 & \text{if } x_i > T_i \\ 0 & \text{otherwise} \end{cases} \tag{19}$$

To learn the threshold values, we apply an additional loss at training time which is minimized when the batchwise mask and the threshold mask are identical.

$$L_{batchwise}(X, T, m) = \sum_{j=1}^{|X|} \sum_{i=1}^{n} (M_{threshold}(x, T)_i - M_{batchwise}(X, m)_{j,i})(X_{j,i} - T_i) \tag{20}$$

G Attention Function

The attention mechanism described in GNMT (Wu et al., 2016) involves a learned "Attention Function" $A(x_i, y_j)$ which takes a "source vector" x_i and a "target vector" y_j, and must be computed for every source time step i and target time step j. In GNMT, the attention function is implemented as a feed forward neural network with a hidden layer of size n. It can be expressed as:

$$A_{GNMT}(x_i, y_j) = \sum_{d=1}^{n} V_d tanh((x_i U)_d + (y_j W)_d) \tag{21}$$

Where U and W are trainable weight matrices and V is a trainable weight vector.

For performance reasons, in our models, we used a slightly different attention function:

$$A(x_i, y_j) = \sum_{d=1}^{n} V_d tanh((x_i U)_d) tanh((y_j W)_d) \tag{22}$$

With our attention function, we can simultaneously compute the attention function on multiple source time steps and multiple target time steps using optimized matrix multiplications. We found little difference in quality between the two functions.